# DevOps (開發維運)
# 自動化建置管理

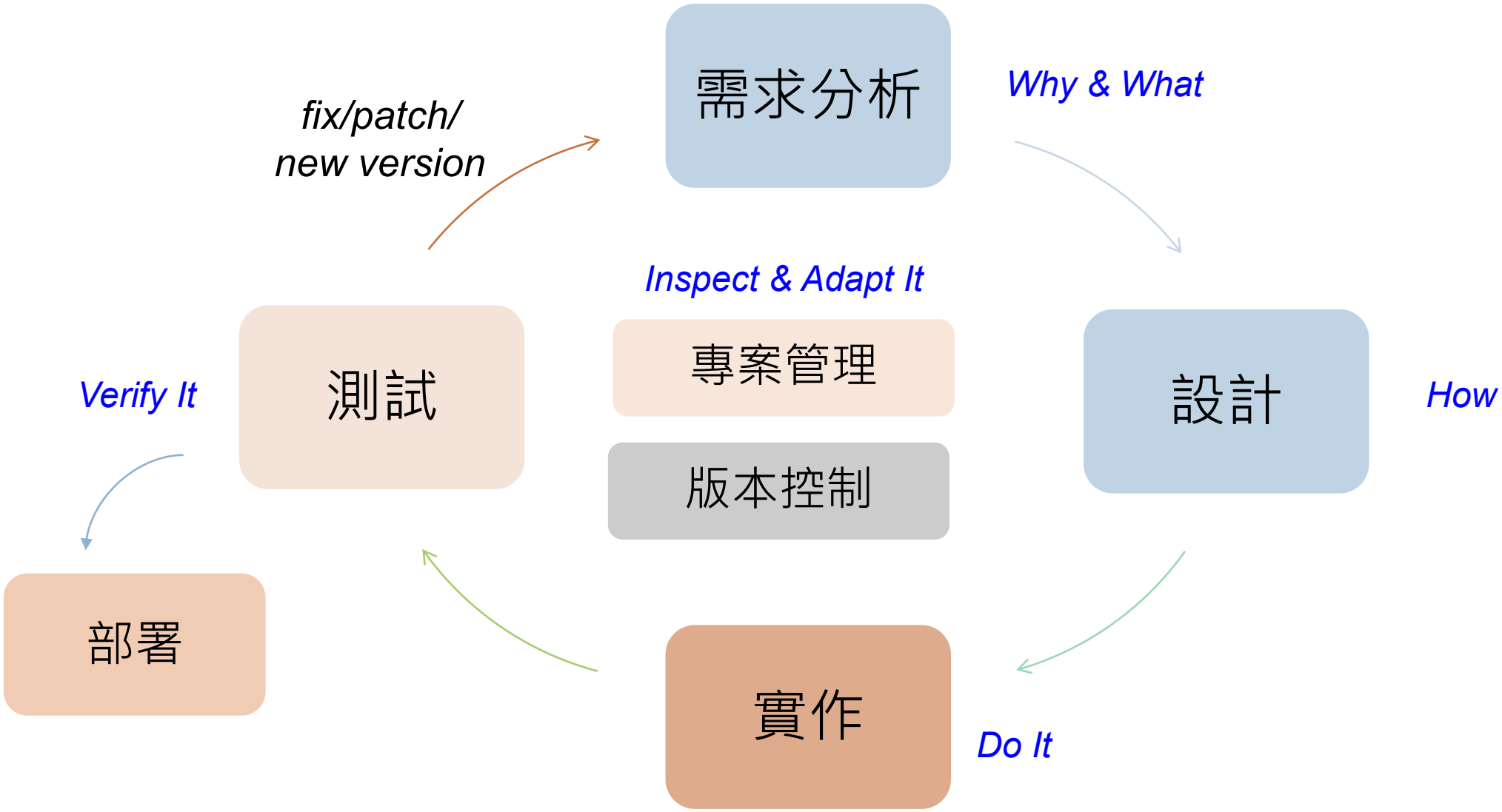
# 單元大綱



- DevOps簡介
- Jenkins與Maven
- SonarQube
- GitHub Actions

# 軟體開發的主要步驟

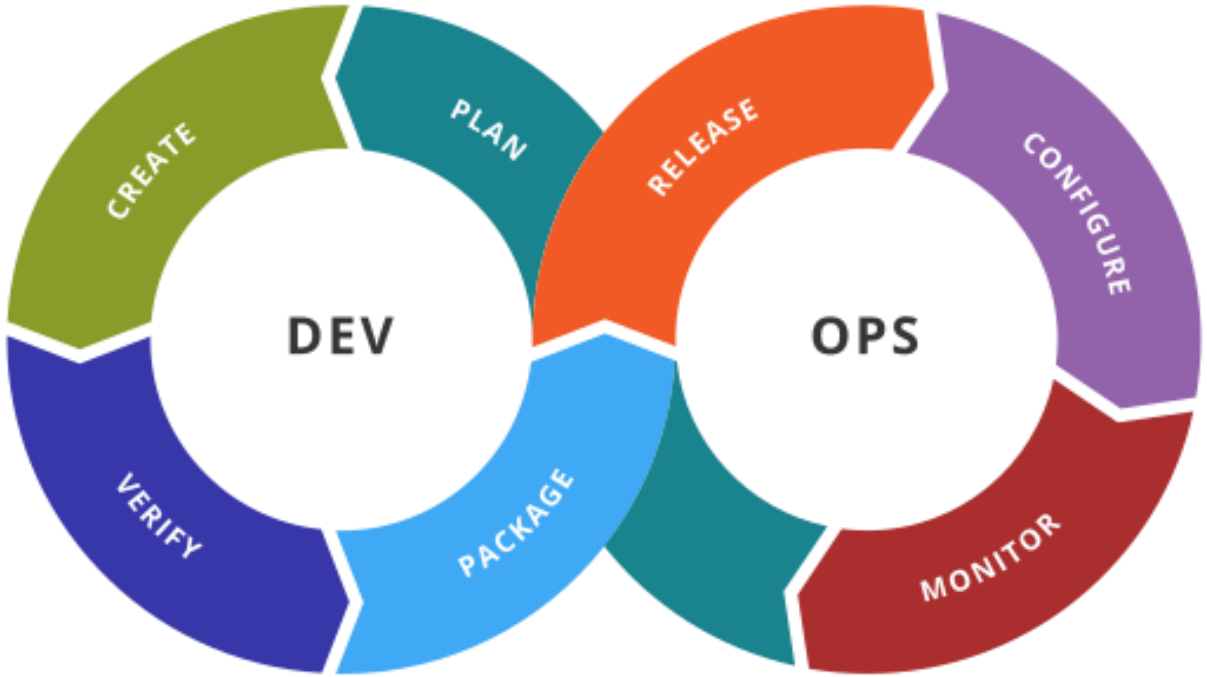



需求分析
Why & What
設計
How
實作
Do It
測試
Verify It
部署
fix/patch/
new version
Inspect & Adapt It
專案管理
版本控制

# 什麼是DevOps？



- Development + Operations ，也就是開發與維運。



# DevOps要解決什麼問題？1

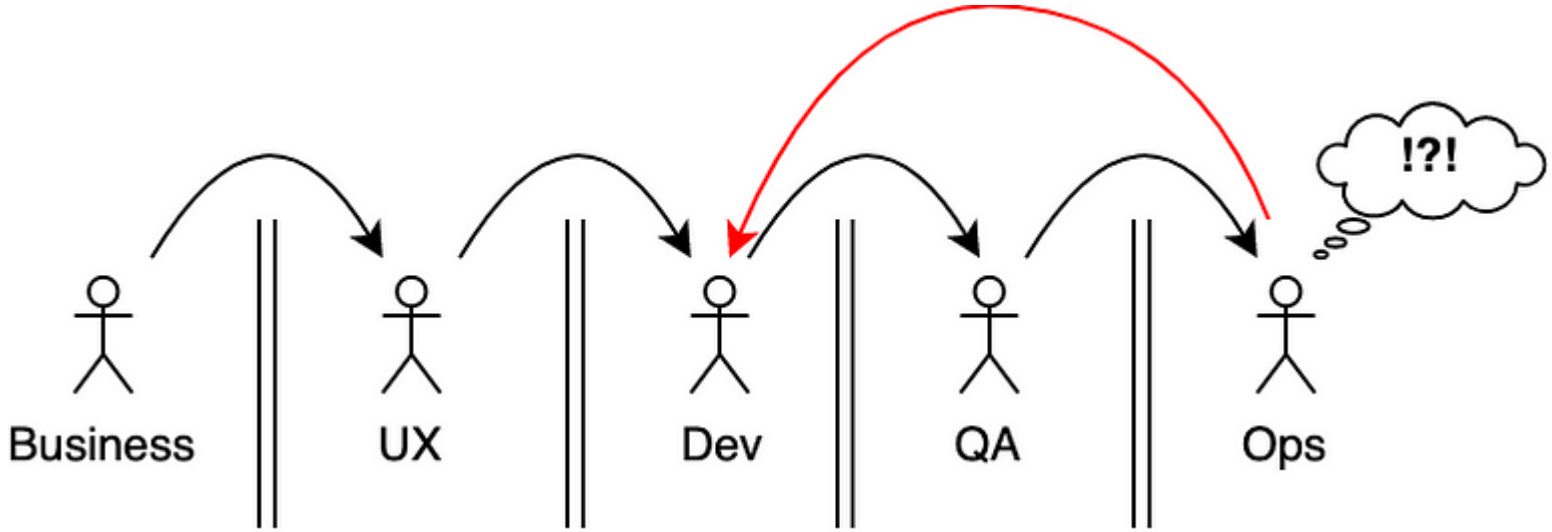


- 企業組織開發產品目標
  - 面對多種使用者、具備多樣功能。
  - 頻繁交付產品、部署週期短。
- 開發和維運紛爭
  - 開發人員為滿足使用者需求，不斷更新版本，不管硬體網路部署問題，增加維運人員工作量。
  - 維運人員為成功部署，可能變更設定，造成上線責任歸屬紛爭。

# DevOps要解決什麼問題？2



- 開發與維運團隊間可能產生「困惑牆(Wall of Confusion)」
  - https://levelup.gitconnected.com/the-wall-of-confusion-623057a4dd26



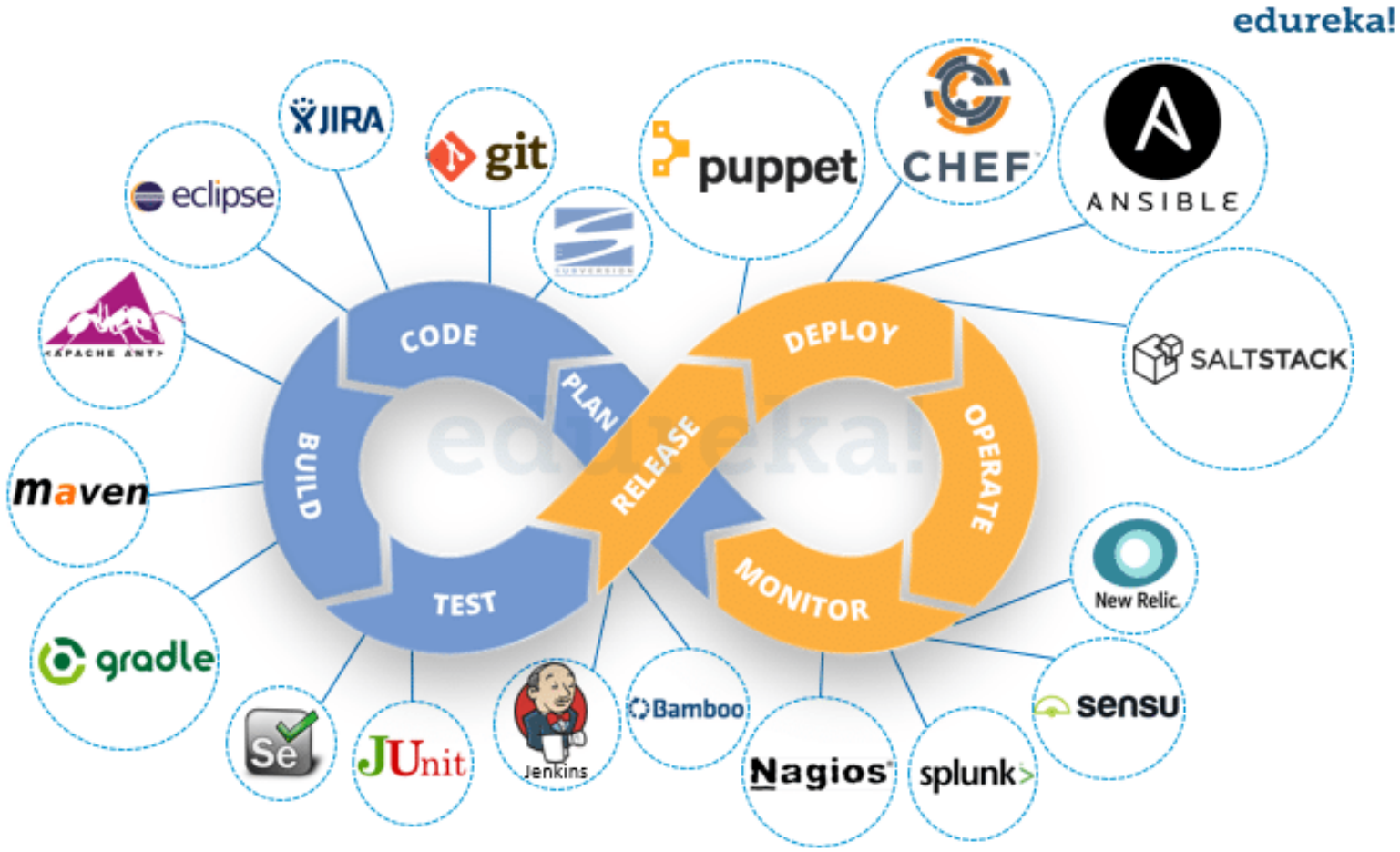
# DevOps的目標



- DevOps要解決開發和維運整合的問題
  - 提高軟體品質
  - 讓開發有效率
  - 避免重工
- DevOps的主要優勢是速度：
  - 從發現需求到產品上線的時間越短，能得到的回饋與市場也就越大。

# DevOps工具






https://www.edureka.co/blog/devops-tools

# DevOps可能運作流程



- 提交 (Commit)：程式在本地端開發機測試後，提交至遠端版本控制系統。
- 建置 (Build)：持續整合系統Jenkins CI，偵測到版本更新，自動從 Git 取得最新程式進行建置與測試。。
- 單元測試：Jenkins 完成建置後，自動執行指定的單元測試。
- 進行品質檢測/安全檢測：如運用SonarQube找到主要品質問題與Security、Reliability、Maintainability之可改善議題。
- 部署到測試環境 (Deploy to Test Environment)：完成單元測試後，Jenkins 將程式部署到跟應用環境 (Production Environment) 相近的驗收測試環境進行測試。
- 驗收測試：在驗收測試環境進行自動化功能測試，例如 Selenium 測試。開發人員或客戶手動進行跟實際情況類似的驗收測試。
- 部署到應用環境 (Deploy to Production Environment)：通過所有測試後，將最新版本部署到實際應用環境。

# Jenkins1



- 由Java編寫的開源持續整合工具。
- 執行在Servlet容器中(例如Apache Tomcat)。
- 它支援軟體組態管理(SCM, Software Configuration Management)工具，如Git、 Subversion 、ClearCase等。
- 可執行基於Apache Maven、Apache Ant、Gradle的專案。
- 可執行任意的Shell指令碼和Windows批次處理命令。

https://ntouedu-my.sharepoint.com/:v:/g/personal/a0250_o365_ntou_edu_tw/EbqnqbYURCxOr_xdsqQQDfcBJoPUo522u8iFEAnR0egEvA

# Jenkins2



- Jenkins是一個開源的自動化伺服器，用於實現持續整合(Continuous Integration, CI)和持續交付(Continuous Delivery, CD)流程。
  - 可自動化各種任務，例如建構、測試、部署和發布，使開發和交付過程更加有效率與可靠。

# Jenkins特點



- 開源：Jenkins是開源軟體，可免費使用。
- 插件(Plug-in)系統：Jenkins擁有豐富的插件庫，可以擴展其功能，支持各種工具和技術。
- 可擴展性：Jenkins可以透過Master-Slave架構來擴展，以處理大型專案。
- 支持多種作業系統：Jenkins可以在Windows、Mac和Linux上運行。
- 強大的社群：Jenkins擁有活躍的使用者和開發者社群，提供豐富的資源和支持。

# Jenkins安裝1



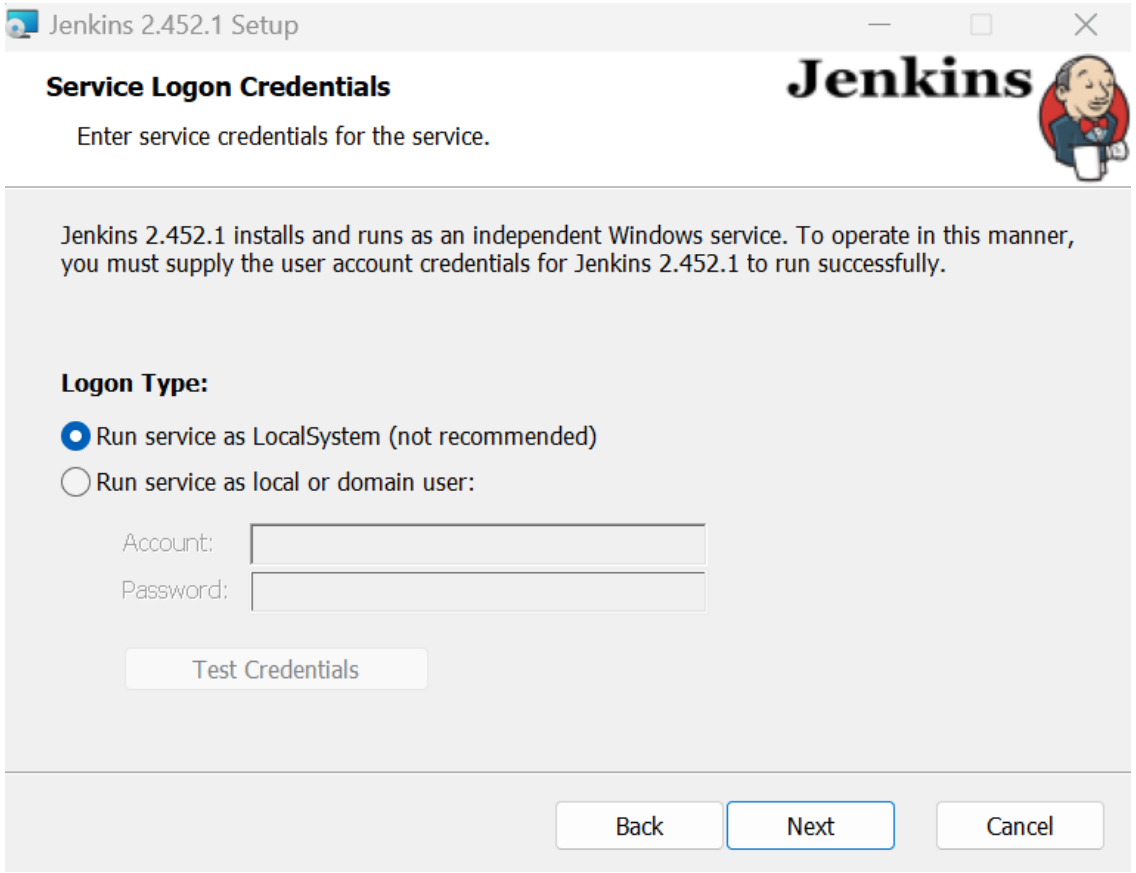
- 安裝Jenkins:
  - https://www.jenkins.io/doc/book/installing/
  - Windows:
    - https://www.jenkins.io/doc/book/installing/windows/
    - https://www.jenkins.io/download/thank-you-downloading-windows-installer-stable/
- 安裝過程中會需要解鎖：

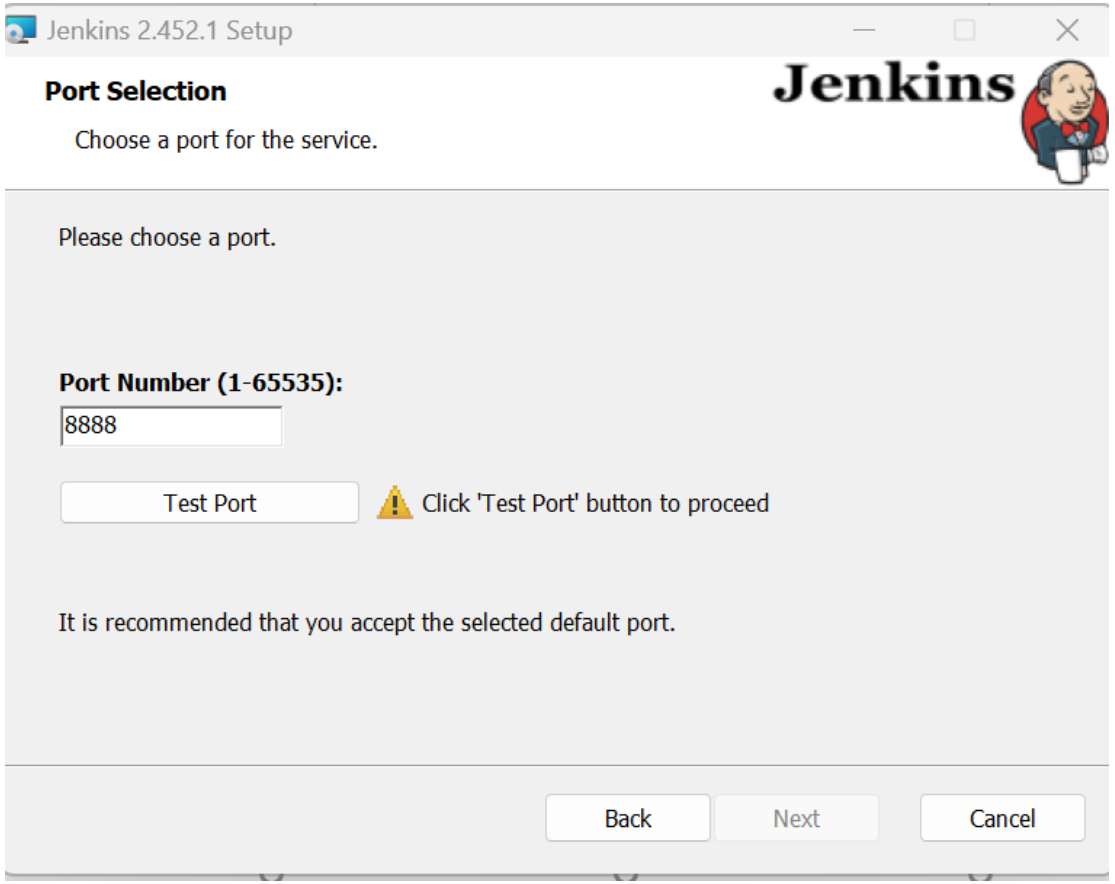



- 需先安裝Java JDK：
  - https://www.oracle.com/tw/java/technologies/downloads/

# Jenkins安装2




Jenkins 2.452.1 Setup
Service Logon Credentials
Enter service credentials for the service.
Jenkins
Jenkins 2.452.1 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.452.1 to run successfully.
Logon Type:
Run service as LocalSystem (not recommended)
Run service as local or domain user:
Account:
Password:
Test Credentials
Back
Next
Cancel


Jenkins 2.452.1 Setup
Port Selection
Choose a port for the service.
Jenkins
Please choose a port.
Port Number (1-65535):
8888
Test Port
Click 'Test Port' button to proceed
It is recommended that you accept the selected default port.
Back
Next
Cancel

# Jenkins安裝3

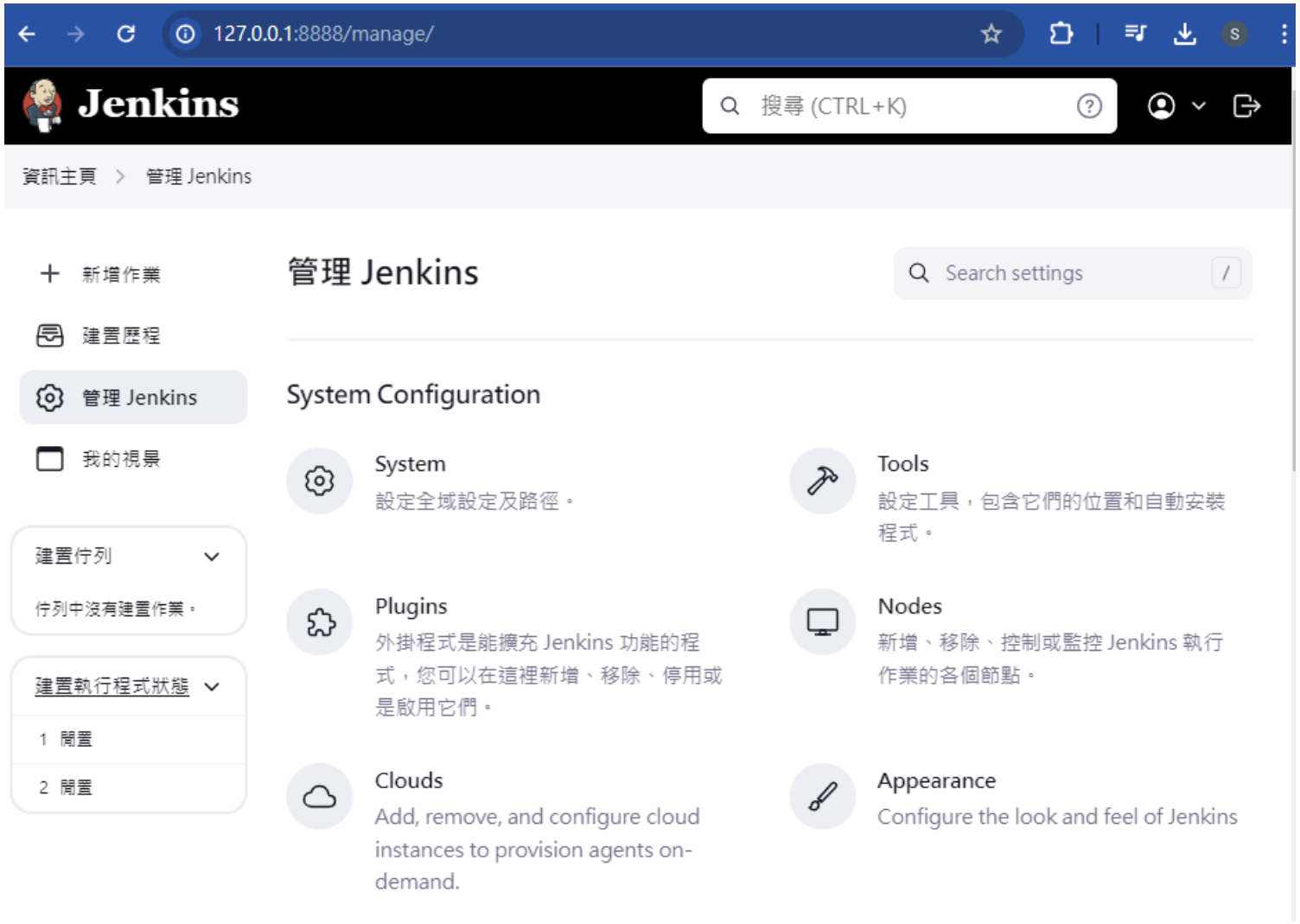



127.0.0.1:8888/login?from=%2F
開始使用
解鎖 Jenkins
為了確保 Jenkins 是由管理員安全地安裝，已將密碼寫入記錄檔（不確定在哪裡可以找到？）此檔案位於伺服器上：
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
請從其中一個位置複製密碼並貼到下方。
管理員密碼
繼續


開始使用
自訂 Jenkins
外掛為 Jenkins 擴充額外的功能以滿足不同的需求。
安裝推薦的外掛
安裝 Jenkins 社群認為最有用的外掛。
選擇外掛來安裝
選擇並安裝最適合您的外掛。

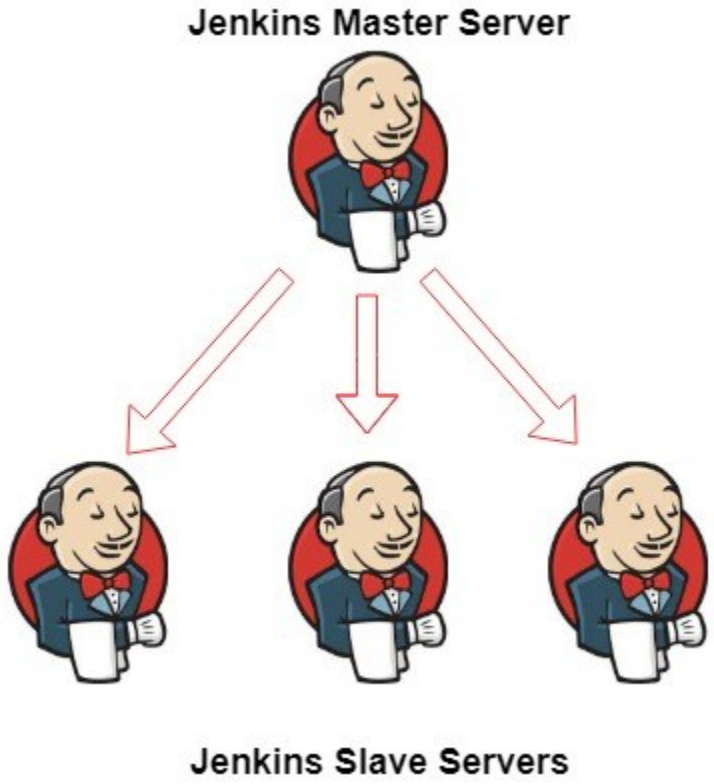
# Jenkins安裝4




127.0.0.1:8888/manage/
Jenkins
搜尋 (CTRL+K)
資訊主頁 > 管理 Jenkins
新增作業
建置歷程
管理 Jenkins
我的視景
建置佇列
佇列中沒有建置作業。
建置執行程式狀態
1 閒置
2 閒置
管理 Jenkins
Search settings
System Configuration
System
設定全域設定及路徑。
Tools
設定工具，包含它們的位置和自動安裝程式。
Plugins
外掛程式是能擴充 Jenkins 功能的程式，您可以在這裡新增、移除、停用或是啟用它們。
Nodes
新增、移除、控制或監控 Jenkins 執行作業的各個節點。
Clouds
Add, remove, and configure cloud instances to provision agents on-demand.
Appearance
Configure the look and feel of Jenkins

# Jenkins Architecture[1]






https://labs.tadigital.com/index.php/2018/07/16/jenkins-master-and-slave-configuration/

# Jenkins Architecture2



- Master
  - 排程執行工作 (Jobs)
  - 發送工作命令給 Slave
  - 監控所有 Slaves 與記錄執行結果
  - 亦可以自己執行工作
- Slave
  - 只負責執行 Master 發送過來的工作指令

# Jenkins-Free-Style1



- 先選擇新增作業->輸入任意項目(專案)名稱->選擇[建置 Free-Style 軟體專案]->點選確定。
- 接下來可以設定：
  - General
  - 原始碼管理
    - 可設定GitHub網址
  - 建置觸發程序
    - 如[Poll SCM]即可設定每分鐘輪詢GitHub一次(*/1 * * * *)
  - Build Steps
  - 建置後動作

# Jenkins-Free-Style2



- 我們先設定“Build Steps”，再選擇[執行Shell] (Linux)或[執行Windows 批次指令](Windows)



# Jenkins-Free-Style3



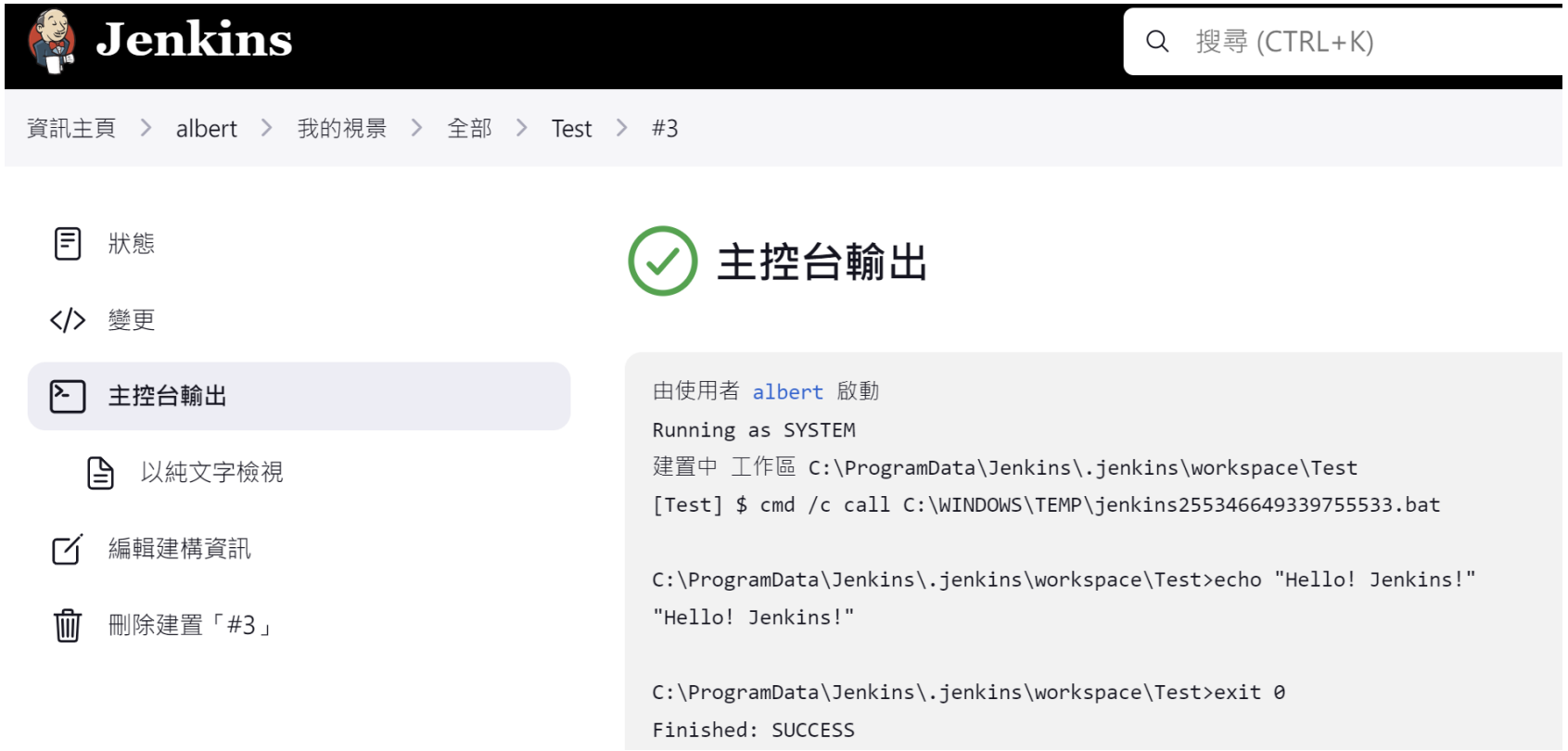
- 可於首頁的[資訊首頁]找到並執行此工作(作業)



# Jenkins-Free-Style4



- 可點進單次build的“主控台輸出”頁籤，觀看執行的結果。



# 可透過Jenkins進行的任務



- 以Java專案為例，我們可以做的事情包含：
  - 拉相依性套件
  - 編譯程式
  - 執行測試案例(單元測試、效能測試、端到端測試)
    - JUnit、JMeter、Selenium等
  - 執行品質檢測
    - https://docs.sonarqube.org/latest/analyzing-source-code/scanners/jenkins-extension-sonarqube/
  - 執行安全檢測
    - https://plugins.jenkins.io/dependency-check-jenkins-plugin/

# Jenkins Pipeline



- Jenkins Pipeline允許使用者以程式碼的方式定義和管理建置、測試和部署流程。
- Jenkins Pipeline的主要目的是提供一種統一的方式來描述整個軟體開發過程中的自動化步驟，並且這些步驟可以被版本控制。



(https://wck.org/relief/chefs-for-india)

# Declarative Pipeline



- Declarative Pipeline是一種基於簡單、易讀的語法，用於定義整個自動化流程。
  - 它提供了高層次的抽象，使得不熟悉腳本語言(如Groovy)的使用者也能夠輕鬆定義流水線。

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                // Build code here
            }
        }
        stage('Test') {
            steps {
                // Run tests here
            }
        }
        stage('Deploy') {
            steps {
                // Deploy code here
            }
        }
    }
}
```

# Scripted Pipeline



- Scripted Pipeline是使用Groovy腳本語言的流水線語法，提供了更高的靈活性和精細的控制。
  - 可以使用腳本語言的所有功能，進行更複雜的自動化流程設計。

```
node {
    // 定義環境變數
    def buildVersion

    stage('Preparation') {
        // 在這裡執行一些前置準備工作
        echo 'Preparing for the build'

        // 生成建置版本號
        buildVersion = sh(script: 'date "+%Y%m%d%H%M%S"',
returnStdout: true).trim()
    }

    stage('Build') {
        // 進行建置(使用先前生成的版本號)
        echo "Building version ${buildVersion}"
    }
```

# Jenkins Pipeline: Stage View



- Stage View是Pipeline標準插件，用於提供一個視覺化的方式來查看和監控Pipeline的執行。
- Stage View將Pipeline 切分為多個階段(stages)，每個階段均代表重要步驟，如建置、測試、部署等。
  - 這有助於了解整體Pipeline與每個階段的執行狀態。



# SonarQube



- SonarQube是一個開源的平台，用於持續檢查和測量原始碼的品質。
  - 它支援多種程式語言，並提供詳細的原始碼品質報告，幫助開發者識別和解決潛在的問題。
  - 可整合至Jenkins或GitHub Action，作為Pipeline中的一個步驟。

# 安裝SonarQube1



- 下載SonarQube
  - 前往SonarQube 官方網站下載最新版本的SonarQube。
    - https://www.sonarsource.com/products/sonarqube/downloads/
- 解壓縮
  - 將下載的SonarQube安裝包解壓縮到你想要的位置。
- 啟動SonarQube
  - 在命令列中，進入SonarQube目錄，然後根據你的作業系統執行命令：

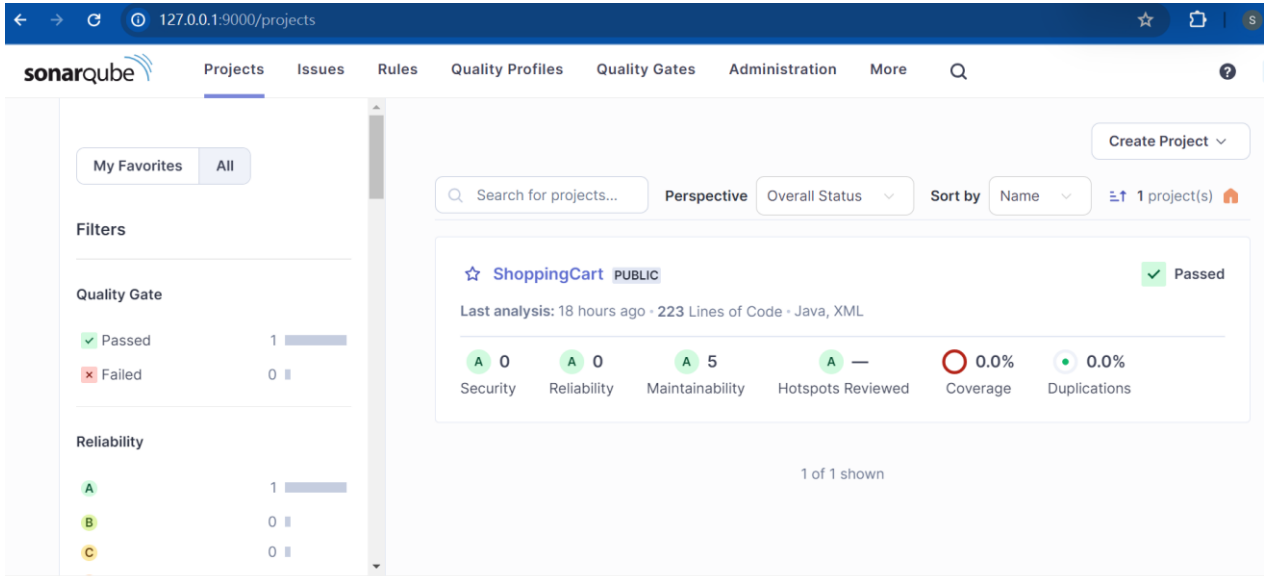
    *Windows: bin/StartSonar.bat*

    *Linux: bin/sonar.sh start*

# 安裝SonarQube2



- 檢查SonarQube是否運行
  - 打開瀏覽器，瀏覽http://localhost:9000，應該能夠看到SonarQube的主頁。
- 初次登入時，預設的管理員帳號和密碼都是admin。
  - 登錄後應立即更改預設密碼



# 於Maven配置SonarQube



- 可在Maven設定中配置SonarQube
  - 在 *Maven安裝目錄/conf/settings.xml* 找到Maven的settings.xml文件，加入右方片段。

*<your_token>要替換成您的token：*
*右上角Icon -> My Account -> Security -> 在Generate Tokens的表單上輸入任意的Name(如MyTask)，選擇[Global Analysis Token] -> [Generate]，即能獲得token。*

```
<settings>
 …
  <pluginGroups>
    <pluginGroup>org.sonarsource.scanner.maven</pluginGroup>
  </pluginGroups>
  <profiles>
    <profile>
      <id>sonar</id>
      <activation>
        <activeByDefault>true</activeByDefault>
      </activation>
      <properties>
        <sonar.host.url>http://localhost:9000</sonar.host.url>
        <sonar.login>your token</sonar.login >
      </properties>
    </profile>
  </profiles>
  <activeProfiles>
    <activeProfile>sonar</activeProfile>
  </activeProfiles>
</settings>
```

# 安裝SonarQube Scanner



- 下載SonarQube Scanner
  - 前往SonarQube Scanner官方網站下載最新版本的SonarQube Scanner。
    - https://docs.sonarsource.com/sonarqube/9.8/analyzing-source-code/scanners/sonarscanner/
- 解壓縮
  - 將下載的SonarQube Scanner安裝包解壓縮到你想要的位置。
- 設定環境變數
  - 將SonarQube Scanner的bin目錄新增到你的系統PATH中
- 檢查安裝
  - 在終端或命令提示字元中，輸入以下命令檢查安裝是否成功：
    *sonar-scanner -v*

# 執行SonarQube Scanner



- 於專案目錄下新增 sonar-project.properties 檔案：

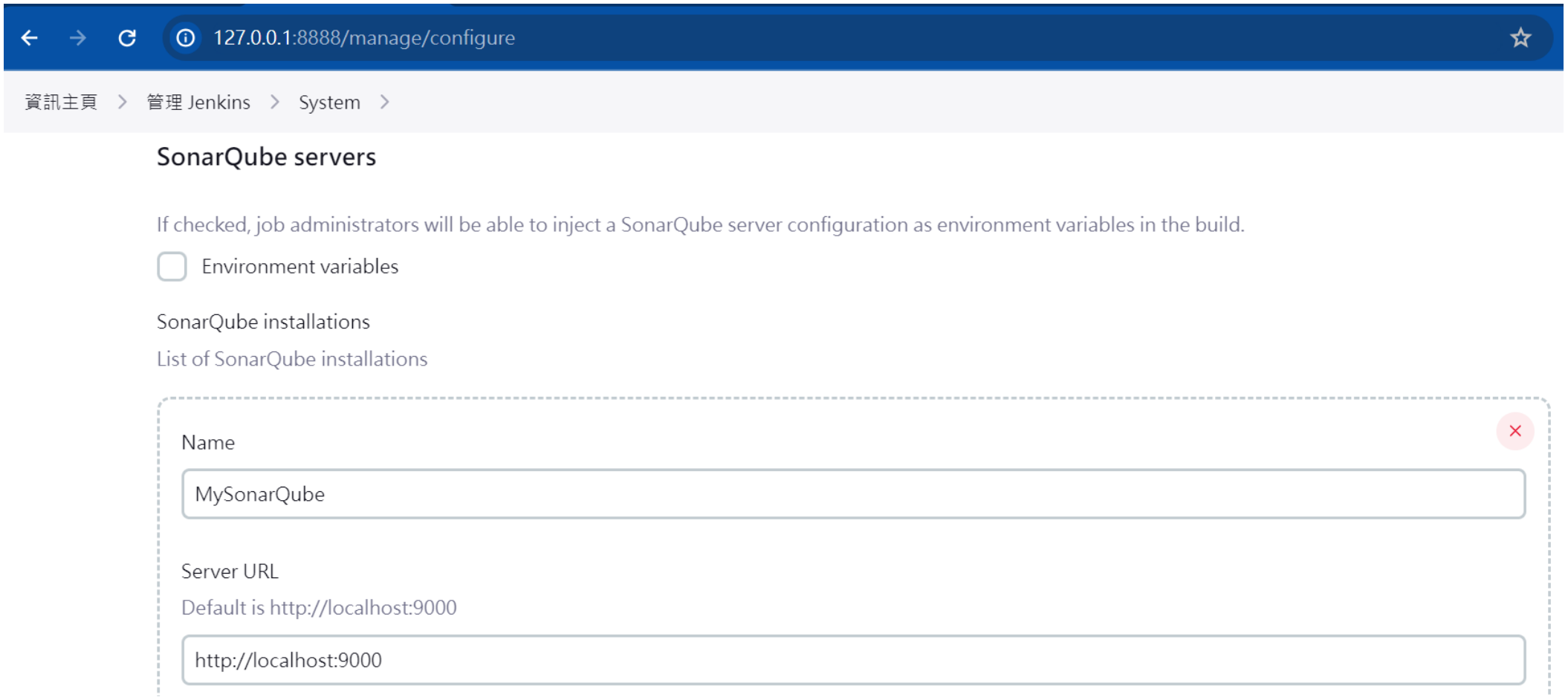
```
sonar.projectKey=SonarScannerTest
sonar.sources=./src/main/java
sonar.java.binaries=./target/classes
sonar.host.url=http://localhost:9000
sonar.login=<your token>
```

- 於專案目錄下執行底下指令：

```
sonar-scanner
```

# 於Jenkins中設定SonarQube Server






http://127.0.0.1:8888/manage/configure (可能需要安裝插件)

# 於Pipeline中加入SonarQube Scan任務



- 可將SonarQube Scan任務加入專案的Pipeline

```
stage('SonarQube Scan') {
        steps {
            withSonarQubeEnv('MySonarQube') {
                bat "mvn sonar:sonar -Dsonar.token=your_token"
            }
        }
    }
```

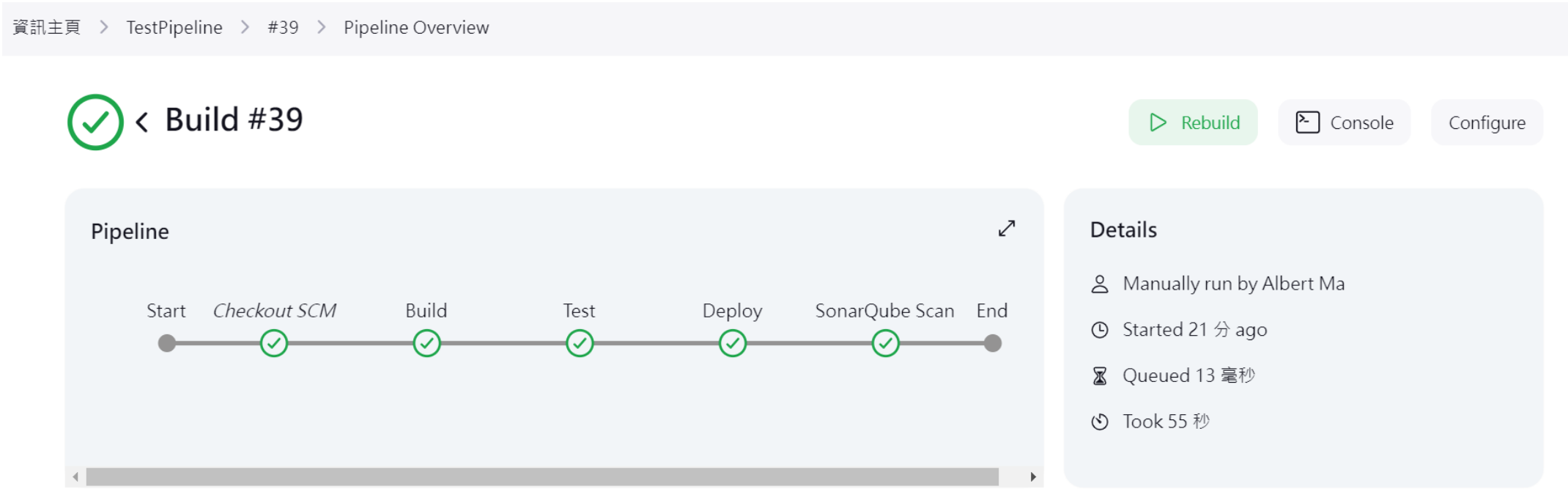
- 若maven已經配置好token，可直接下無參數指令

```
stage('SonarQube Scan') {
        steps {
            withSonarQubeEnv('MySonarQube') {
                bat "mvn sonar:sonar"
            }
        }
    }
```

# Pipeline執行會包含SonarQube Scan

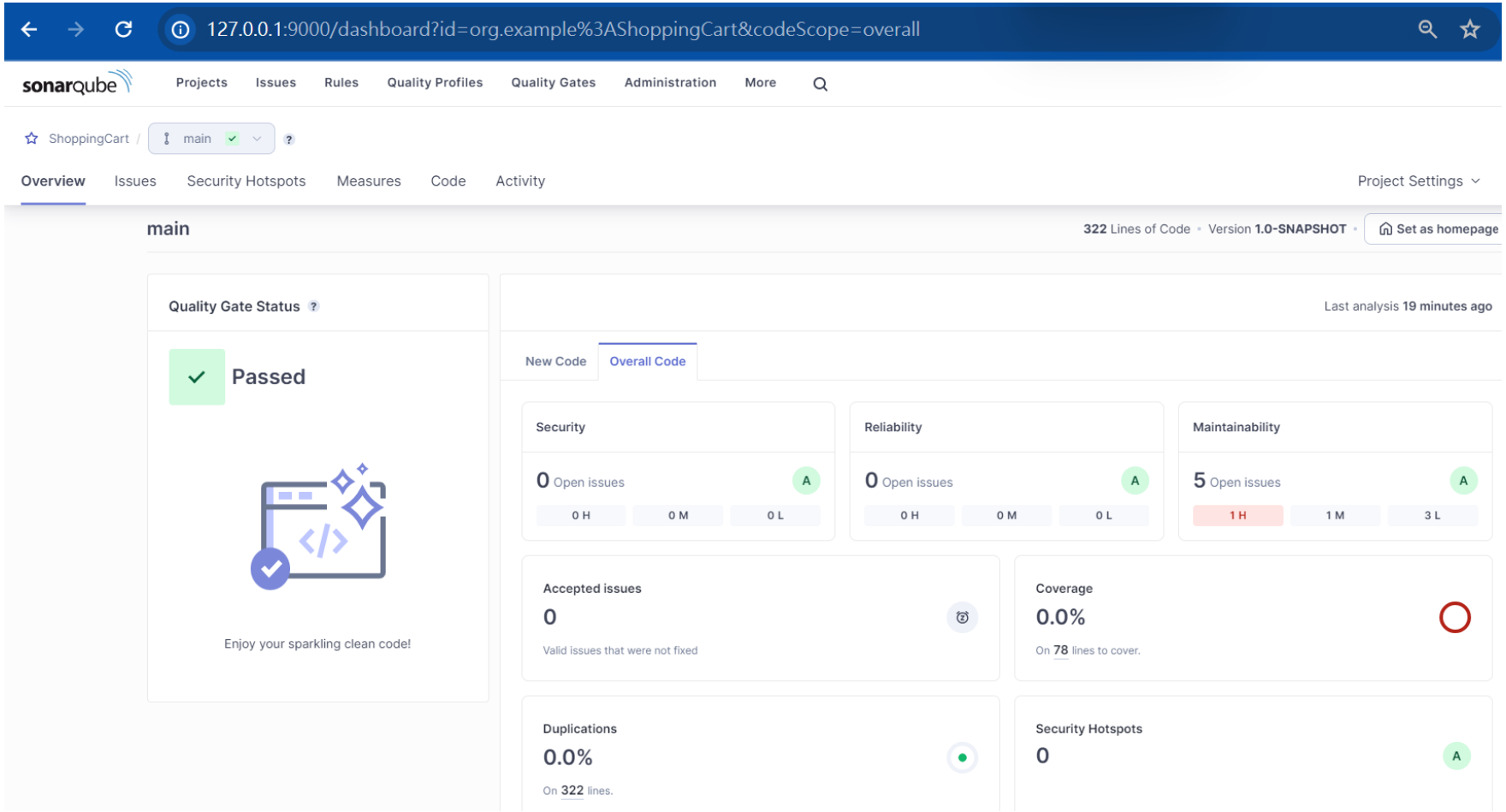


- 所有Pipeline步驟皆可以自動化地循序執行



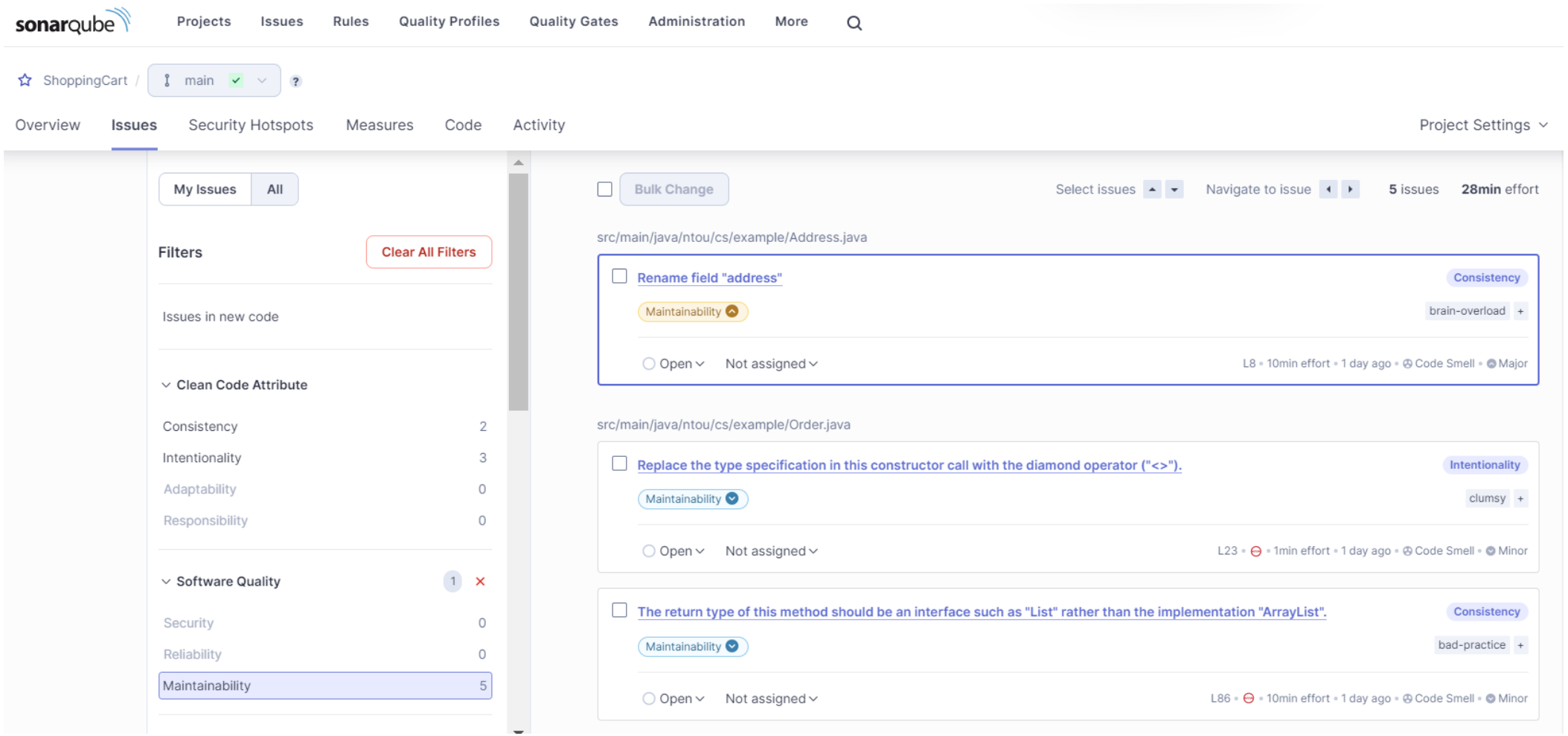
# SonarQube Server會呈現Scan結果





# SonarQube Server會呈現Scan後的建議





# GitHub Actions[1]



- GitHub Actions可自動化軟體工作流程
  - 直接從 GitHub 建構、測試和部署程式碼。
  - 讓程式碼審查、分支管理和問題分類按照需求運作。

# GitHub Actions2

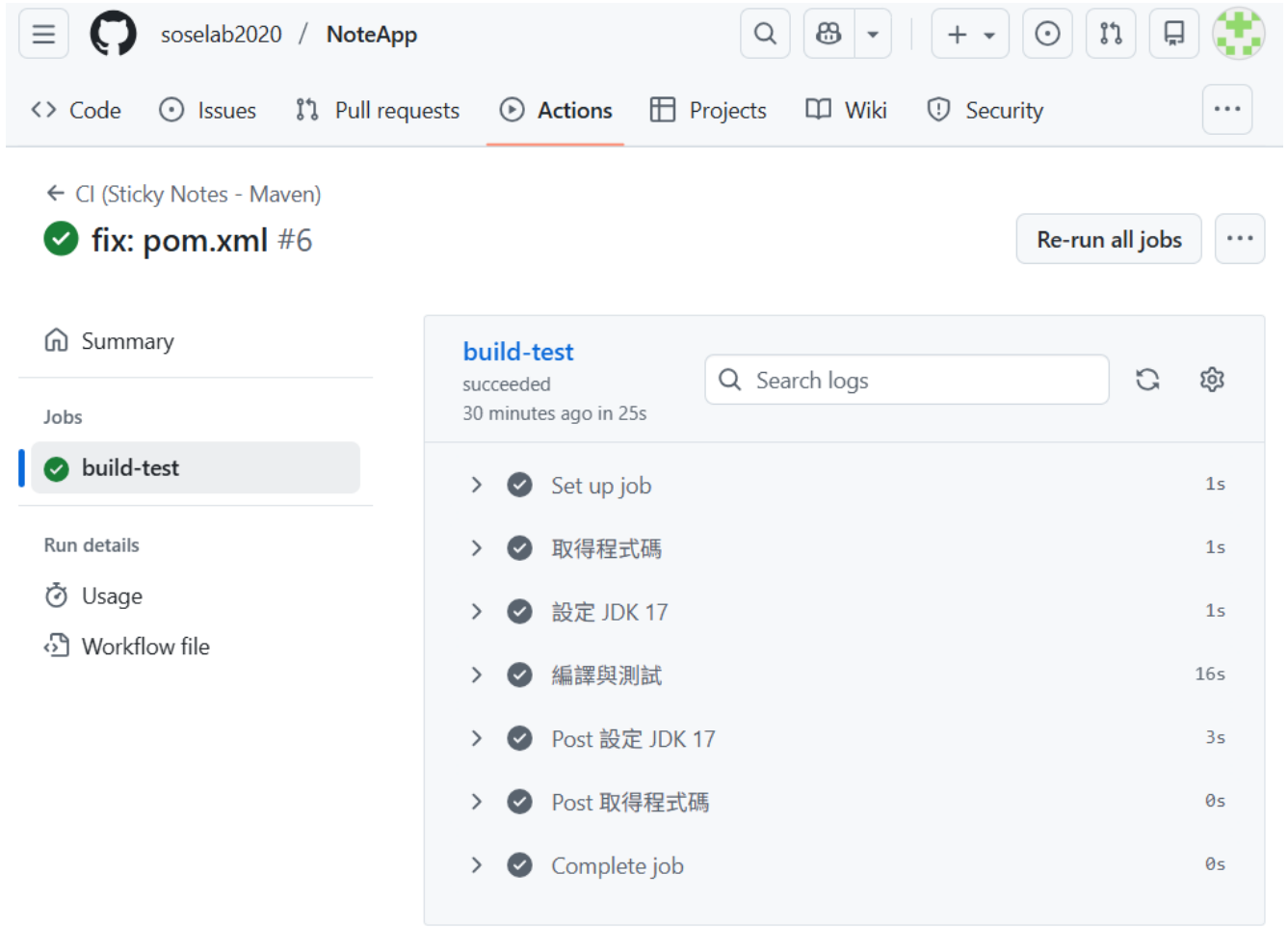


- Event: 觸發自動化工作流程的功能。
- Job: 是在同一個 Runner 執行的一組 Step。
- Step: 在 Job 內可以執行命令獨立工作，Step 可以是一個命令或一個 Action。
- Action: 獨立的執行內容，可以自行撰寫，也可以使用 GitHub 社群創作的 Action 來完成工作流程
- Runner: 負責執行自動化程式的伺服器，以容器(Docker)運作在 Ubuntu Linux、Microsoft Windows 和 macOS 上

# GitHub Actions3



- 以GitHub Actions執行MockMVC測試範例：




https://github.com/soselab2020/NoteApp/actions

# Any Question?



